

OTC: Optimal Tool Calls via Reinforcement Learning

Hongru Wang^α, Cheng Qian^β, Wanjun Zhong^δ, Xiusi Chen^β, Jiahao Qiu^σ,
Shijue Huang^μ, Bowen Jin^β, Mengdi Wang^σ, Kam-Fai Wong^α, Heng Ji^β

^αThe Chinese University of Hong Kong, ^βUniversity of Illinois Urbana-Champaign

^σPrinceton University, ^δSun Yat-sen University, ^μHong Kong University of Science and Technology
hrwang, kfwong@se.cuhk.edu.hk, hengji@illinois.edu

Abstract

Tool-integrated reasoning (TIR) augments large language models (LLMs) with the ability to invoke external tools, such as search engines and code interpreters, to solve tasks beyond the capabilities of language-only reasoning. While reinforcement learning (RL) has shown promise in improving TIR by optimizing final answer correctness, existing approaches often overlook the efficiency and cost associated with tool usage. This can lead to suboptimal behavior, including excessive tool calls that increase computational and financial overhead, or insufficient tool use that compromises answer quality. In this work, we propose Optimal Tool Call-controlled Policy Optimization (OTC-PO), a simple yet effective RL-based framework that encourages models to produce accurate answers with minimal tool calls. Our method introduces a tool-integrated reward that jointly considers correctness and tool efficiency, promoting high tool productivity. We instantiate this framework within both Proximal Policy Optimization (PPO) and Group Relative Preference Optimization (GRPO), resulting in OTC-PPO and OTC-GRPO. Experiments with Qwen-2.5 and Qwen-Math across multiple QA benchmarks show that our approach reduces tool calls by up to 73.1% and improves tool productivity by up to 229.4%, while maintaining comparable answer accuracy. To the best of our knowledge, this is the first RL-based framework that explicitly optimizes tool-use efficiency in TIR ¹.

1 Introduction

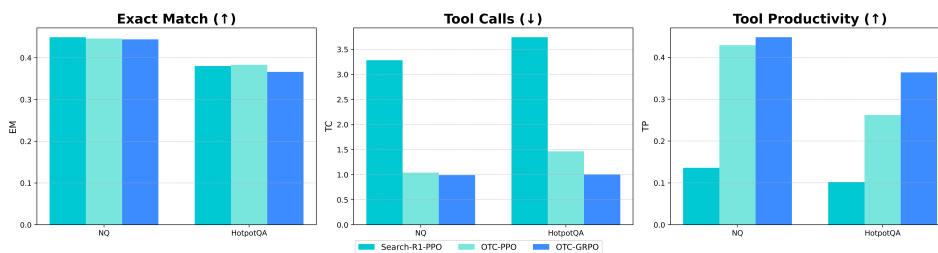


Figure 1: The performance of OTC-PO v.s. Search-R1 on Qwen2.5-7B-Base model. Our OTC achieves similar final performance but much higher tool calling efficiency comparing with the original method.

Recent advances in large language models (LLMs) have demonstrated impressive reasoning capabilities when fine-tuned via reinforcement learning (RL) with simple rule-based rewards. This has led

¹This work is ongoing; we look forward to releasing further details soon.

to the emergence of large reasoning models (LRMs) - such as OpenAI’s o1 [1], DeepSeek-R1 [2], and QwQ [3] - which exhibit creative and diverse behaviors in extended Chain-of-Thought (CoT) reasoning [4]. These models develop meta-reasoning capabilities reminiscent of human cognition, such as reflection and decomposition, which have been shown to enhance performance on complex tasks [5–7]. However, the internal reasoning capacity of LRMs alone is often insufficient or even infeasible for tasks that require interaction with external environments, such as accessing up-to-date domain knowledge or performing precise computations. To overcome these limitations, tool-integrated reasoning (TIR) has emerged as a promising paradigm [8–12], enabling LLMs to interface with external tools like search engines, calculators, or code interpreters. By bridging language understanding with external capabilities, TIR significantly expands the problem-solving scope of LLMs beyond language-only reasoning.

While recent efforts have sought to extend reinforcement learning (RL) to tool-integrated reasoning (TIR) by leveraging rule-based rewards tied to final answer correctness [13–15], these approaches fall short in guiding models to use tools judiciously. Despite achieving strong performance and improved generalization, current methods often encourage indiscriminate tool usage, introducing critical challenges during both training and inference. On the one hand, frequent and unnecessary tool calls incur substantial computational and temporal overhead, as executing tools—especially at scale—demands significant infrastructure and increases operational costs, limiting deployment feasibility. We also observe a pattern of *cognitive offloading* [16], in which the model extensively relies on external tools, thereby limiting the development and utilization of its internal reasoning capabilities. This issue becomes more pronounced as model size increases. On the other hand, optimizing solely for answer accuracy neglects the inherent costs (e.g., time, financial) and effectiveness of individual tool usage, which may lead to inefficient reasoning behaviors. In particular, models may excessively rely on external tools even when fewer or no tools are actually needed (i.e., *Tool Overuse*) [11], or conversely, avoid using tools altogether even when external assistance is crucial (i.e., *Tool Underuse*) [17]. Both patterns hinder optimal reasoning and highlight the need for more refined training objectives - ones that balance correctness with efficient and contextually appropriate tool usage.

We aim to develop such a method based on a foundational assumption: for *each question* and *each model*, there exists an optimal number of tool calls, defined as the minimal number required for the model to arrive at a correct answer. Crucially, this optimal number is not fixed across models or questions. For the same question, models with different reasoning or representation capabilities may require different levels of tool assistance. Similarly, even for a single model, the number of necessary tool calls can vary significantly depending on the complexity of different question. For example, some questions may be solvable through internal reasoning alone - requiring zero tool calls² - while others may necessitate one or more tool interactions to yield a correct answer. This observation highlights the limitations of supervised fine-tuning (SFT), which typically enforces a uniform policy across diverse model-question pairs. In contrast, reinforcement learning (RL) provides a more flexible framework, enabling reward signals to be dynamically adjusted based on each model’s behavior and tool usage. Ideally, among all tool-integrated reasoning trajectories that lead to the correct answer for the same question, the reward function should assign the highest reward to the trajectory that uses the optimal (i.e., minimal) number of tool calls. Other correct trajectories that involve excessive tool calls - by definition, suboptimal - should be rewarded less³. This formulation allows the model to learn policies that jointly optimize for both answer correctness and efficient, context-aware tool use on a per-question, per-model basis.

We propose Optimal Tool Calls controlled Policy Optimization (OTC-PO), a simple yet effective RL-based method that enables large language models (LLMs) to learn precise and adaptive control over external tool usage. OTC-PO trains models to optimize the number of tool calls required to reach a correct solution, thereby reducing both training and inference costs without sacrificing accuracy. To achieve this, we introduce a tool-integrated reward that modulates traditional reward signals, such as correctness, with a scaling coefficient reflecting tool efficiency. This encourages the model to prioritize correct answers that require fewer tool invocations. Our formulation shifts the optimization objective from correctness alone to tool productivity, defined as the ratio between task benefit (e.g., answer accuracy) and tool usage cost (e.g., number of tool calls). To the best of our knowledge, this is the first RL-based framework that explicitly optimizes for both the efficiency and effectiveness of tool-integrated reasoning. OTC-PO is lightweight and broadly applicable, requiring only minimal changes

²In such cases, the task reduces to pure language-only reasoning.

³All such trajectories incur unnecessary tool usage and are thus less efficient.

to standard RL pipelines - just a few lines of code - making it easy to adopt in existing systems. We validate our approach across two common tool modalities: web search and code execution. Using multiple LLMs—including Qwen-2.5-3B/7B-Base and Qwen2.5-Math-1.5B/7B-Base, we show that OTC-PO significantly reduces tool calls during reasoning trajectories while maintaining comparable accuracy to conventional approaches. In summary, the key contributions are as follows:

- We are *the first* to i) systematically address the tool efficiency problem via RL, which has been largely overlooked in prior research; ii) identify the cognitive offloading phenomenon in TIR of LLMs; iii) introduce the concept of tool productivity to measure the effectiveness and efficiency of TIR.
- We propose a simple, scalable, and generalizable OTC-PO algorithm to encourage the model to use optimal tool calls to solve the problem and maximize the tool productivity, building on top of a fundamental observation that there is an optimal number of tool calls for each question and model pair. We note that it is compatible with various RL algorithms and can be easily implemented with only a few line changes.
- We implement OTC-PPO and OTC-GRPO as two typical methods without losing the adaptability and generalization, and the experimental results on several benchmarks and baselines demonstrate significant reductions in tool call cost while preserving most of the accuracy in both in-domain and out-of-domain evaluation.

2 Related Work

Tool Utilization for LLMs. Teaching LLMs to use tools enables them to interact with external environments while overcoming several inherent limitations such as restricted access to up-to-date or domain-specific knowledge and poor mathematical operation capabilities. There are three major methods which can achieve this goal: 1) prompting engineering [18–23], which guides the model’s behavior through carefully designed input templates or few-shot examples without modifying model weights; 2) supervised finetuning on tool-integrated reasoning datasets [24, 8, 25, 26, 11], where the model learns from annotated trajectories that demonstrate when and how to use tools in context; and 3) reinforcement learning [13, 14, 27, 28], which allows the model to directly learn tool-use strategies by interacting with an environment and optimizing long-term rewards, enabling more adaptive and goal-directed behaviors. While prompting and supervised fine-tuning have shown promising results, they rely heavily on expert-designed prompts and tool-integrated annotations, which limits their scalability and generalization. As a result, recent efforts have shifted toward reinforcement learning, which demonstrates improved performance and generalization through simple rule-based rewards.

Tool-integrated Reasoning Efficiency. Only few of studies from prompting engineering and supervised fine-tuning attention on tool-integrated reasoning efficiency issues in terms of the cost of tool usages during the reasoning [23, 17, 11]. In detail, Wang et al. [23] first propose a prompting-based framework: Self Divide-and-Conquer (SelfDC) to leverage the self-aware confidence score of LLMs to decide whether or not need to call tools during reasoning, achieving better trade-off between effectiveness and efficiency in the context of RAG. Furthermore, several works follow this direction and explore more boarder applications and larger tool spaces [11, 17, 29]. For example, SMART [11] collect the well-designed dataset to finetune the model to only call tools when the knowledge is outside the inherent parametric knowledge of LLMs. Despite these advancements, most existing approaches still rely on complex prompt engineering or annotated datasets, which hinders their adaptability and scalability to new scenarios. In contrast, the efficiency of tool-integrated reasoning within reinforcement learning frameworks remains largely underexplored.

Reward Shaping in RL. Reward shaping plays a critical role in reinforcement learning for tool-using language models, as it directly shapes the behavior the model learns to optimize [30, 31]. Recent advances have introduced several reward signals for LLMs to consider the correctness and the language-only reasoning efficiency together such as the length of responses and difficulty of the questions [32–34]. For instances, Length Controlled Policy Optimization (LCPO) [32] is proposed to satisfy the length constraints while optimizing reasoning performance and some efforts try to dynamically allocate inference time compute based on task complexity [33, 35].

3 Methodology

In this section, we first provide a definition of task considering the both effectiveness and efficiency of tool-integrated reasoning, followed by general RL framework and then our proposed Optimal Tool Call-controlled Policy Optimization (OTC-PO).

3.1 Task Definition

Given a question q , and an environment $\mathcal{E}$ that provides access to a set of tools $\mathcal{T} = \{t_0, t_1, \dots, t_n\}$, the language model $\mathcal{M}$ can *optionally* interact with the environment by calling specific tools in $\mathcal{T}$, obtaining the corresponding tool results from $\mathcal{E}$, and iteratively repeating this processing until the final answer is driven. Without losing generalization, the tool-integrated reasoning trajectory τ_k at step k is defined as follows:

$$\tau_k = (r_0, tc_0, o_0), (r_1, tc_1, o_1), \dots, (r_k, tc_k, o_k), \quad (1)$$

where r_i, tc_i, o_i denotes the reasoning, tool call and returned observation respectively. Importantly, we also account for reasoning steps that do not involve tool usage. Suppose the step p does not need to call tools, then the tc_p and o_p become empty string, the reasoning content r_p can either be merged with the subsequent reasoning step r_{p+1} to form the new r_{p+1}^* , or, if p is the last step⁴, be used directly to derive the final answer. The objective of the task is to generate the correct answer $\hat{a}$ with the minimal cost of tools in the full trajectory τ as follows:

$$\arg \min_{\tau} \text{Cost}(\tau) \quad \text{subject to} \quad \mathcal{M}(q, \tau) = \hat{a}, \quad (2)$$

Here the cost is measured as the number of tool calls within the trajectory τ considering its simplicity and generalization. Thus the model is encouraged to not only generate correct answer but also minimize the tool calls.

3.2 Tool-integrated Reinforcement Learning

Inspired by recent success to use RL for better reasoning in LLMs, several efforts try to extend RL to tool-integrated reasoning with the objective functions can be defined as follows:

$$\max_{\pi_{\theta}} \mathbb{E}_{q \sim \mathcal{D}, y \sim \pi_{\theta}(\cdot | q; \mathcal{E})} [r_{\phi}(q, y)] - \beta \mathbb{D}_{kl} [\pi_{\theta}(y | q; \mathcal{E}) \parallel \pi_{\text{ref}}(y | q; \mathcal{E})], \quad (3)$$

where $\pi_{\theta}, \pi_{\text{ref}}$ stand for the policy model and reference model respectively, r_{ϕ} is the reward function and $\mathbb{D}_{kl}$ is the KL-divergence measure. q is the question drawn from the dataset $\mathcal{D}$ and y is the generate outputs consists of the tool-integrated reasoning trajectory τ and the final answer a . To optimize this goal, there are two well-established policy-gradient RL methods: Proximal Policy Optimization (PPO) and Group Relative Policy Optimization (GRPO).

PPO in TRL. Proximal Policy Optimization (PPO) is a widely used policy-gradient method in lots of tasks [36]. Given our formulation, PPO updates the policy, using trajectories sampled from the previous policy, and maximizes the following objective:

$$\mathcal{J}_{\text{PPO}}(\theta) = \mathbb{E}_{q \sim \mathcal{D}, y \sim \pi_{\text{old}}} \left[\frac{1}{\sum_{t=1}^{|y|} \mathbb{I}(y_t)} \sum_{t=1}^{|y|} \mathbb{I}(y_t) \cdot \min \left(\rho_t A_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) A_t \right) \right], \quad (4)$$

where π_{θ} and π_{old} are current and previous policy models, and $p_t = \frac{\pi_{\theta}(y_t | q, y_{<t}; \mathcal{E})}{\pi_{\text{old}}(y_t | q, y_{<t}; \mathcal{E})}$. Here, $\mathbb{I}(y_t)$ is an indicator function marking whether token y_t is generated by the model (i.e., r_i and tc_i) or returned from the environment $\mathcal{E}$ (i.e., o_i). The advantage estimate A_t is computed via Generalized Advantage Estimation (GAE) and ϵ a PPO clipping threshold to constrain the policy update.

⁴The last step is practically determined by the predefined maximal tool calls.

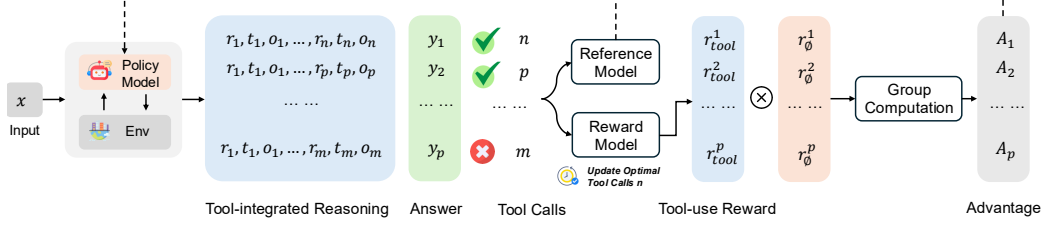


Figure 2: An overview of OTC-GRPO Algorithm.

GRPO in TRL. To improve the stability of policy optimization in language models and avoid reliance on an additional value function approximation, Group Relative Policy Optimization (GRPO) is introduced which uses the relative ranking of multiple sampled outputs as a baseline for computing advantages, rather than fitting a value function. For each input question q , GRPO samples a group of G response $\{y_1, y_2, \dots, y_G\}$ from the reference policy π_{ref} . The trained policy π_θ is then updated by maximizing the following objective:

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}_{q \sim \mathcal{D}, \{y_i\}_{i=1}^G \sim \tau_{old}(\cdot|q; \mathcal{E})} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{\sum_{t=1}^{|y_i|} \mathbb{I}(y_i, t)} \sum_{t=1}^{|y_i|} \mathbb{I}(y_i, t) \cdot \min \left(p_t \hat{A}_{i,t}, \text{clip} \left(p_t, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_{i,t} \right) \right] - \beta \mathbb{D}_{KL}[\pi_\theta \parallel \pi_{ref}], \quad (5)$$

Here, $p_t = \frac{\pi_\theta(y_{i,t}|x, y_{i,<t}; \mathcal{R})}{\pi_{old}(y_{i,t}|x, y_{i,<t}; \mathcal{R})}$, $\hat{A}_{i,t}$ denotes the advantage at token t in response y_i , computed based on the relative ranking of rewards within the group, β controlling the KL-regularization strength. The clipping threshold ϵ ensures stable updates.

Reward Design. Reward signals serve as the optimization objective and directly guide the behavior of the policy model during training. Most prior predominantly focuses on rule-based correctness rewards [13, 14], as illustrated below:

$$r_\phi(q, y) = r_{correct} = 1 \text{ if } a = \hat{a} \text{ else } 0, \quad (6)$$

where a is the extracted predicted answer from the response y and $\hat{a}$ is the ground truth answer. Some of them further consider the format reward by assigning additional reward besides the correctness such as r_{format} if the model generate the response in the required format. We discuss this in Appendix.

3.3 OTC-PO: Optimal Tool Call-controlled Policy Optimization

Instead of only considering the correctness (and optionally format) of the answer, we argue it is important to consider the effectiveness and efficiency of tool-integrated reasoning trajectory. Specifically, different questions may require different numbers of tool interactions depending on the internal capabilities of model, and for each question-model pair, there exists an optimal (i.e., minimal) number of tool calls needed to reach the correct answer. Given the real-world cost and latency associated with tool calls, it is crucial to move beyond coarse, answer-only rewards and instead provide finer-grained signals that capture the overall quality of TIR trajectory.

To this end, we propose Optimal Tool Call-controlled Policy Optimization (OTC-PO), can be easily integrated with any RL algorithms, and is broadly applicable across different models and settings, aiming to preserve overall accuracy while significantly reducing the cost of tool interactions. At the heart of OTC-PO is a reward mechanism that ranks the current number of tool calls m relative to the optimal number of tool calls n given the question and model. In most realistic settings, the true optimal number n is unknown in advance. To address this, OTC-PO approximates n by tracking the minimal number of tool calls observed across different trajectories to arrive the correct answer for the same question. This approach enables the model to learn efficient tool use without requiring prior knowledge of the optimal tool budget. Nonetheless, the framework can be naturally extended

to scenarios where the optimal tool call number is known a priori (e.g., $n = 0$ for language-only reasoning). We instantiate OTC-PO in two variants: OTC-PPO and OTC-GRPO, tailored to different underlying RL algorithms.

OTC-PPO. Since there is no multiple trajectories for same question q in single optimization step, we design the tool reward according to the number of tool calls m in the current trajectory as follows:

$$r_{tool} = \cos\left(\frac{m * \pi}{2m + c}\right) \quad (7)$$

where c is the smooth constant that controls the reward decay rate. The smaller the value, the faster the punishment and the more it encourages less use of tools; the larger the value, the more tolerant it is. In this way, among trajectories, those requiring more tool calls will receive lower rewards than those achieved with fewer tool calls. Although we can track the m here for approximation of n in later steps, we empirically find it requires much longer training steps due to poor sampling efficiency in PPO.

OTC-GRPO. We first identify the trajectories $\{\tau^1, \tau^2, \dots, \tau^p\}$ leading to correct answer from the group for the question q , and get the corresponding number of tool calls for each trajectory: $\mathcal{C} = \{c^1, c^2, \dots, c^p\}$, and we can calculate the minimal tool calls $k = \min(\mathcal{C})$, serving as the *local* approximation of optimal tool calls for q . Furthermore, we can update k during multiple epochs to approximate the *global* optimal tool calls if the policy model finds the better solution with less than k calls in later iterations. We use n to indicate the approximation of optimal tool calls, and m the number of tool calls in the current trajectory for q . Therefore we design the reward as follows:

$$r_{tool} = \begin{cases} 1 & \text{if } f(m, n) = n = 0 \\ \cos\left(\frac{m * \pi}{2m + c}\right) & \text{if } n = 0 \\ \sin\left(\frac{f(m, n) * \pi}{2n}\right) & \text{otherwise} \end{cases} \quad f(m, n) = \begin{cases} 0, & \text{if } m = 0 \text{ and } n = 0 \\ m, & \text{if } n = 0 \\ \frac{2nm}{m + n}, & \text{otherwise} \end{cases} \quad (8)$$

where $f(m, n)$ is the mapping function to re-map the m to the range from 0 to $2n$. Briefly, the key here is to assign the highest reward 1 (a.k.a, $\sin(\pi/2)$) when the policy model achieves optimal tool calls n , and when actual number of tool calls m deviates from n , either exceeding or falling short, the model receives a comparatively lower reward⁵. Thus, we can assign different rewards dynamically according to the different n and m for the question q .

Tool-integrated Reward Design. Inspired by recent study [33], we regard the tool-integrated reward r_{tool} as a *coefficient* of conventional reward function $r_\phi(q, y)$, leading to the final tool-integrated reward function:

$$r_\phi^{tool}(q, y) = \alpha * r_{tool} * r_\phi(q, y) \quad (9)$$

where α is a hyperparameter that controls the scaling of the tool reward as r_ϕ is usually same for correct or wrong answer. Notably, this multiplicative structure ensures that tool efficiency is mainly rewarded when the primary task objective (e.g., answer correctness) is satisfied. For example, when the final answer is incorrect, r_ϕ is 0, effectively disabling the influence of r_{tool} . This design helps mitigate reward hacking by preventing the policy model from being incentivized to use tools without ultimately producing a correct answer. If the answer is correct, r_ϕ is 1 and then r_{tool} begins to dominate the scale of the reward, thus encouraging less tool usage. We highlight this design is non-trivial since there are several advantages of this design: i) It maximally preserves overall accuracy, with a theoretical justification [33]; ii) It mitigates the risk of reward hacking compared to additive forms (i.e., $r_{tool} + r_\phi$), which we found to be unstable and sub-optimal⁶; iii) It generalizes well to different formulations of r_ϕ , such as $r_\phi = r_{correct}$ or $r_\phi = r_{correct} + r_{format}$. We provide the theoretical justification, more explanations and examples in the Appendix.

⁵We provide the reward illustration in the Appendix for better understanding.

⁶We empirically found that the model achieves higher reward by call less tools without correct answer if we use addition operation.

Models	NQ			HotpotQA		
	EM ($\uparrow$)	TC ($\downarrow$)	TP ($\uparrow$)	EM ($\uparrow$)	TC ($\downarrow$)	TP ($\uparrow$)
Qwen2.5-3B(-Base)						
R1-Base	0.226	-	-	0.201	-	-
SFT	0.249	-	-	0.186	-	-
RAG	0.348	1.0	0.348	0.255	1.0	0.255
IRCoT	0.111	10.0	0.011	0.164	10.0	0.016
Search-R1-PPO	0.403	1.738	0.232	0.279	1.716	0.163
OTC-PPO	0.355	1.010 (∇ 41.9%)	0.351 (∇ 51.3%)	0.260	1.026 (∇ 40.2%)	0.253 (∇ 55.2%)
OTC-GRPO	0.444	1.008 (∇ 42.0%)	0.440 (∇ 89.7%)	0.365	1.387 (∇ 19.2%)	0.263 (∇ 61.3%)
Qwen2.5-7B(-Base)						
R1-Base	0.270	-	-	0.242	-	-
SFT	0.318	-	-	0.217	-	-
RAG	0.349	1.0	0.349	0.299	1.0	0.299
IRCoT	0.224	9.999	0.022	0.133	9.982	0.013
Search-R1-PPO	0.449	3.282	0.136	0.380	3.741	0.102
OTC-PPO	0.446	1.040 (∇ 68.3%)	0.429 (∇ 215.4%)	0.383	1.464 (∇ 60.9%)	0.262 (∇ 156.9%)
OTC-GRPO	0.444	0.990 (∇ 69.8%)	0.448 (∇ 229.4%)	0.366	1.005 (∇ 73.1%)	0.364 (∇ 256.9%)

Table 1: The results of OTC-PO with different baselines in search. The results except Search-R1 are directly copied from original paper [13]. We highlight the relative improvements compared with the Search-R1 in terms of TC and TP. It is clear that our method achieves comparable performance with much lower TC and higher TP.

4 Experiments

4.1 Set up

Datasets and Baselines. We mainly follow the Search-R1 [13] and ToRL [14] setting and use same baselines and datasets for the fair and comprehensive evaluation. Specifically, we use NQ and HotpotQA as training dataset for search, and we use the dataset provided in ToRL for code. We also directly compare our methods with several baselines such as SFT, Base-RL, retrieval-augmented generation baselines (i.e., RAG, IRCoT) and more importantly the Search-R1 and ToRL themselves.

Evaluation Metrics. Besides the exact match (EM) and the average tool calls (TC), we additionally define a new metric, *tool productivity* (TP), which measures the effectiveness and efficiency of tool calls during inference. Specifically, TP is defined as the number of correctly answered questions per unit of tool call: $TP = \frac{\sum_{i=1}^N \mathbb{I}\{y_i = \hat{y}_i\}}{\sum_{i=1}^N tc_i}$ ⁷, where $\mathbb{I}$ is the indicator function that equals 1 if the predicted answer $\hat{y}_i$ matches the ground truth y_i , and tc_i denotes the number of tool calls used in the i_{th} instance. This metric reflects how efficiently the model converts tool usage into correct answers, capturing both utility and cost in a single measure. While EM provides a standard measure of accuracy, it does not reflect the underlying reasoning cost. Therefore, we consider TP as more informative indicators of agentic reasoning efficiency – highlighting not only whether the model can provide correct answer, but how economically it arrives at that correctness.

Implementation Details. We re-use the same parameter in Search-R1 [13] and ToRL [14] respectively. There are only minor modification we make to suit our method: i) We slightly change the template in Search-R1 to tell the model that it only need to call tolls when necessary (Appendix), and we do not change the template in ToRL; ii) We set the max turn (maximal number of tool calls) in Search-R1 (ToRL) to 4 (3) to better study the effects of our methods with multiple tool calls are allowed. We set c as corresponding max turns or maximal tool limits, α as 1. We conduct our experiments on 8 A100-80G GPU and re-produce the results of Search-R1 and ToRL independently due to the different value of maximal tool calls in their original setting. We implement OTC-GRPO using the global approximation of optimal tool calls since it leads to more stable and better optimization.

4.2 Main Results

Search as Tool. Table 1 shows the results for search. There are several key insights can be drawn: i) OTC-PPO achieves significant improvement in terms of TC and TP compared with Search-R1, and OTC-GRPO can further boost the performance due to more accurate approximation, resulting in

⁷This can also be understood as the fraction between benefits and cost.

Models	AIME24			AIME25		
	EM (↑)	TC (↓)	TP (↑)	EM (↑)	TC (↓)	TP (↑)
Qwen2.5-Math-1.5B(-Base)						
Qwen2.5-Math-1.5B-Ins	10.0	-	-	10.0	-	-
Qwen2.5-Math-1.5B-Ins-TIR	13.3	-	-	13.3	-	-
ToRL	26.7	2.0	13.4	23.3	2.3	10.1
OTC-GRPO	20.0	0.6	33.3	17.5	1.3	13.5

Table 2: The results of OTC-PO with different baselines in code. We note this is ONLY preliminary results without hyper-parameter probing.

Method	NQ			HotpotQA		
	A	B	C	A	B	C
Qwen2.5-3B(-Base)						
OTC-PPO	87.6	0.17	4.60	88.8	0.53	4.30
OTC-GRPO	89.8	0.25	6.12	78.1	5.96	10.43
Qwen2.5-7B(-Base)						
OTC-PPO	86.6	0.03	6.45	82.6	0.08	8.60
OTC-GRPO	82.0	0.0	7.26	79.9	0.0	8.15

Table 3: Tool behavioral advantage analysis of OTC-PO against Search-R1.

up to a 73.1% reduction in TC and a 229.4% increase in TP; ii) It is found that our method will not sacrifice the accuracy too much especially for larger LLMs, as evidenced by our OTC-PPO achieves comparable EM score with Search-R1 on Qwen2.5-7B model; iii) Different models have different tool use behaviors on different datasets. If we look at the Search-R1, we can find that 3B model tends to use less tool calls compared with 7B model. This reveals a critical issue: as language models scale up, they tend to over-rely on external tool calls unless explicitly penalized for such behavior. This not only exacerbates the problem with larger models but also leads to an underutilization of their inherent reasoning capabilities. The number of TC is also various across the datasets, which relates to many factors such as inherent capabilities of different models (i.e., self-awareness) and complexity of different datasets, leading to varying optimal number of tool calls for each question and model; iv) As the model size increases, the TC and TP get bigger boost no matter in OTC-PPO or OTC-GRPO. We highlight the ii) and iv) are very important since they compose the great potential scalability for our methods as model scales.

Code as Tool. Table 2 shows the results for code. We already observe similar findings as mentioned above and will release more results soon.

5 Analysis

We mainly conduct our analysis using search as a tool in this section and leave code in the Appendix.

5.1 Tool Use Behavior Analysis

Training. Figure 3 shows the training behaviors of different methods. Our method not only achieves comparable results with fewer tool calls and shorter responses, but also enables faster and more efficient training optimization. This is particularly important, as it significantly reduces the time and cost associated with real-time tool interactions during training, both in terms of financial expenses and computational overhead.

Inference. We identify several representative tool-use behavior types of our proposed method: i) the answer is same with less tool calls compared with baseline (A); ii) the answer is same with more tool calls compared with baseline (B); and iii) our method is able to produce the correct answer using fewer tool calls, whereas the baseline fails to answer correctly (C). Table 3 shows the results of our methods compared with Search-R1. Notably, our method achieves same answers with baseline using fewer tool calls in mostly 80% of test cases (i.e., A). When also considering cases where our

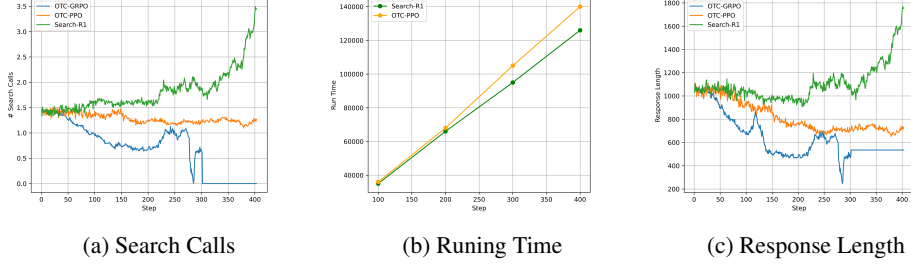


Figure 3: (a): Changes of number of search calls during the training; (b): Runing time analysis between OTC-PPO and Search-R1-PPO; and (c): Change of response length during the training. As the training proceeds, it is clear that our method uses less tool calls with shorter responses and less training time.

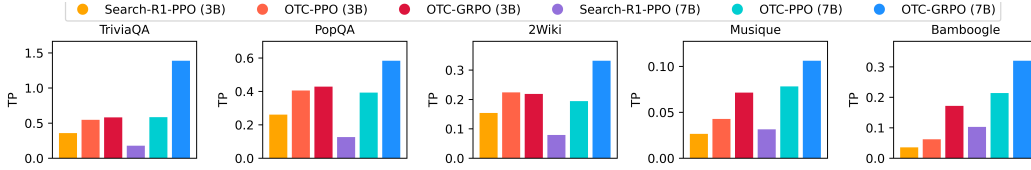


Figure 4: The Out-of-domain performance of OTC-PO and Search-R1 in TP.

Models	TriviaQA		PopQA		2Wiki		Musique		Bamboogle	
	EM ($\uparrow$)	TC ($\downarrow$)	EM ($\uparrow$)	TC ($\downarrow$)	EM ($\uparrow$)	TC ($\downarrow$)	EM ($\uparrow$)	TC ($\downarrow$)	EM ($\uparrow$)	TC ($\downarrow$)
Qwen2.5-3B(-Base)										
Search-R1-PPO	0.566	1.580	0.425	1.631	0.258	1.675	0.051	1.922	0.063	1.766
OTC-PPO	0.551	1.008	0.409	1.009	0.235	1.050	0.045	1.051	0.063	1.016
OTC-GRPO	0.608	1.046	0.441	1.030	0.341	1.561	0.124	1.734	0.266	1.547
Qwen2.5-7B(-Base)										
Search-R1-PPO	0.596	3.353	0.420	3.315	0.326	4.116	0.135	4.294	0.375	3.641
OTC-PPO	0.623	1.066	0.425	1.083	0.363	1.868	0.152	1.942	0.391	1.828
OTC-GRPO	0.597	0.430	0.431	0.739	0.311	0.938	0.130	1.224	0.250	0.781

Table 4: The results of Out-of-Domain (OOD) evaluation of OTC against Search-R1 in EM and TC.

method answers correctly but uses less tools (i.e., C), it demonstrates a favorable balance between effectiveness and efficiency on approximately 90% of test cases across different models and datasets. These results highlight the strength of our reward design in promoting both accuracy and tool-use efficiency.

5.2 Out-of-domain Evaluation

We run out-of-domain evaluation on more QA benchmarks. Figure 4 shows the results of TP and Table 4 shows the results of EM and TC. Generally, we find that our method achieves highest TP across all the model and benchmark. More importantly, we observe that OTC-GRPO is better than OTC-PPO on TP and as the model size increases, our method showcases bigger improvement, serving as another evidence of great scalability of our method. Notably, we find that OTC-PPO outperforms the Search-R1-PPO in terms of EM on all benchmarks with Qwen2.5-7B-Base model with much less tool calls, demonstrating the better generalization capabilities of our methods.

5.3 Case Study

Figure 5 presents a case study to better illustrate the advantages of our proposed OTC-PO. There are several important findings can be drawn from the case study: i) it is evident that Search-R1 issues multiple redundant and unnecessary search calls. This behavior stems from a lack of incentive to minimize tool usage, as the method is not penalized for excessive tool calls. Consequently, it fails to optimize the efficiency and effectiveness of tool calls, resulting in multiple information-poor queries; 2) both OTC-PPO and OTC-GRPO can reduce the number of unnecessary tool calls significantly but they achieve this in different ways. OTC-PPO utilizes more accurate search by generating

Case Study: Trade-off between Reasoning and Tool Use

Question: Are Giuseppe Verdi and Ambroise Thomas both Opera composers ?

Answer: Yes

Search-R1: <think> I need to determine if Giuseppe Verdi and Ambroise Thomas are both opera composers. </think>
<search> Giuseppe Verdi </search>
<observation> ... </observation>
<search> Ambroise Thomas </search>
<observation> ... </observation>
<search> Are Giuseppe Verdi and Ambroise Thomas both Opera composers? </search>
<observation> ... </observation>
<think> Based on the observations, both Giuseppe Verdi and Ambroise Thomas are described as composers, specifically opera composers. </think>
<answer> yes </answer> (**Tool Calls = 3**)

OTC-PPO: <think> First, I need to determine if Giuseppe Verdi and Ambroise Thomas are both opera composers. </think>
<search> Giuseppe Verdi and Ambroise Thomas both Opera composers </search>
<observation> ... </observation>
<think> Based on the search results, Giuseppe Verdi and Ambroise Thomas are both known as opera composers. </think>
<answer> yes </answer> (**Tool Calls = 1**)

OTC-GRPO: To answer the question of whether Giuseppe Verdi and Ambroise Thomas are both Opera composers, let's first reason through the information:
1. Giuseppe Verdi is a well-known Italian opera composer. He is famous for his numerous operas, including "Rigoletto," "La Traviata," and "Aida."
2. Ambroise Thomas is a French opera composer. He is known for his operas such as "Mignon" and "Hamlet."
Based on this reasoning, both Giuseppe Verdi and Ambroise Thomas are indeed Opera composers.
<answer> yes </answer> (**Tool Calls = 0**)

Figure 5: An example that all three methods can provide correct answer but with different cost in terms of reasoning and external tool calls.

information-rich query and OTC-GRPO choose to leverage the internal reasoning capabilities as much as possible since it is encouraged to solve the problem without external tool calls. In summary, *our proposed method not only minimize external tool calls, but also maximize internal reasoning capabilities of LLMs at the same time, resulting in more smart and accurate TIR. This is very important and valuable for almost all agentic reasoning tasks*⁸.

6 Conclusion

In this work, we introduce Optimal Tool Call-controlled Policy Optimization (OTC-PO), a simple yet effective reinforcement learning framework that explicitly encourages language models to generate correct answers with optimal tool calls. Unlike prior work that primarily focuses on final answer correctness, our approach incorporates a tool-integrated reward that accounts for both effectiveness and efficiency of tool usage, thereby promoting tool-use behaviors that are both smart and cost-effective. To the best of our knowledge, this is the first work that explicitly addresses tool-use efficiency in TIR from a RL perspective, and our method offers a scalable and principled solution to make LLMs more capable and economical agents in multiple situations and benchmarks. We emphasize that this is an ongoing project, and we plan to release additional results, analyses, and

⁸More case studies can be found in Appendix.

extensions in future versions. We hope our work can serve as a foundation for further exploration into efficient and scalable tool-integrated reasoning with language models.

Acknowledgement

We thank the open-source community for their invaluable contributions and the spirit of open collaboration that makes research like this possible. In particular, we would like to acknowledge the developers and maintainers of Search-R1, ToRL, VeRL, SandboxFusion, and many others.

References

- [1] OpenAI Team. Openai o1 system card, 2024.
- [2] DeepSeek-AI Team. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025.
- [3] Qwen Team. Qwq-32b: Embracing the power of reinforcement learning, March 2025.
- [4] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2023.
- [5] Hongru WANG, Deng Cai, Wanjun Zhong, Shijue Huang, Jeff Z. Pan, Zeming Liu, and Kam-Fai Wong. Self-reasoning language models: Unfold hidden reasoning chains with few reasoning catalyst. In *Workshop on Reasoning and Planning for Large Language Models*, 2025.
- [6] Yiwei Qin, Xuefeng Li, Haoyang Zou, Yixiu Liu, Shijie Xia, Zhen Huang, Yixin Ye, Weizhe Yuan, Hector Liu, Yanzhi Li, and Pengfei Liu. O1 replication journey: A strategic progress report – part 1, 2024.
- [7] Weihao Zeng, Yuzhen Huang, Qian Liu, Wei Liu, Keqing He, Zejun Ma, and Junxian He. Simplerl-zoo: Investigating and taming zero reinforcement learning for open base models in the wild, 2025.
- [8] Zhibin Gou, Zhihong Shao, Yeyun Gong, yelong shen, Yujiu Yang, Minlie Huang, Nan Duan, and Weizhu Chen. ToRA: A tool-integrated reasoning agent for mathematical problem solving. In *The Twelfth International Conference on Learning Representations*, 2024.
- [9] Chengpeng Li, Mingfeng Xue, Zhenru Zhang, Jiayi Yang, Beichen Zhang, Xiang Wang, Bowen Yu, Binyuan Hui, Junyang Lin, and Dayiheng Liu. Start: Self-taught reasoner with tools, 2025.
- [10] Hongru Wang, Minda Hu, Yang Deng, Rui Wang, Fei Mi, Weichao Wang, Yasheng Wang, Wai-Chung Kwan, Irwin King, and Kam-Fai Wong. Large language models as source planner for personalized knowledge-grounded dialogues. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 9556–9569, Singapore, December 2023. Association for Computational Linguistics.
- [11] Cheng Qian, Emre Can Acikgoz, Hongru Wang, Xiusi Chen, Avirup Sil, Dilek Hakkani-Tür, Gokhan Tur, and Heng Ji. Smart: Self-aware agent for tool overuse mitigation. *arXiv preprint arXiv:2502.11435*, 2025.
- [12] Hongru Wang, Yujia Qin, Yankai Lin, Jeff Z. Pan, and Kam-Fai Wong. Empowering large language models: Tool learning for real-world interaction. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR ’24*, page 2983–2986, New York, NY, USA, 2024. Association for Computing Machinery.
- [13] Bowen Jin, Hansi Zeng, Zhenrui Yue, Dong Wang, Hamed Zamani, and Jiawei Han. Search-r1: Training llms to reason and leverage search engines with reinforcement learning, 2025.
- [14] Xuefeng Li, Haoyang Zou, and Pengfei Liu. Torl: Scaling tool-integrated rl, 2025.

- [15] Mingyang Chen, Tianpeng Li, Haoze Sun, Yijie Zhou, Chenzheng Zhu, Haofen Wang, Jeff Z. Pan, Wen Zhang, Huajun Chen, Fan Yang, Zenan Zhou, and Weipeng Chen. Research: Learning to reason with search for llms via reinforcement learning, 2025.
- [16] Evan F Risko and Sam J Gilbert. Cognitive offloading. *Trends in cognitive sciences*, 20(9):676–688, 2016.
- [17] Yuanhao Shen, Xiaodan Zhu, and Lei Chen. SMARTCAL: An approach to self-aware tool-use evaluation and calibration. In Franck Dernoncourt, Daniel Preoțiuc-Pietro, and Anastasia Shimorina, editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 774–789, Miami, Florida, US, November 2024. Association for Computational Linguistics.
- [18] Cheng Qian, Chi Han, Yi Fung, Yujia Qin, Zhiyuan Liu, and Heng Ji. CREATOR: Tool creation for disentangling abstract and concrete reasoning of large language models. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 6922–6939, Singapore, December 2023. Association for Computational Linguistics.
- [19] Hongru Wang, Rui Wang, Boyang Xue, Heming Xia, Jingtao Cao, Zeming Liu, Jeff Z. Pan, and Kam-Fai Wong. AppBench: Planning of multiple APIs from various APPs for complex user instruction. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 15322–15336, Miami, Florida, USA, November 2024. Association for Computational Linguistics.
- [20] Cheng Qian, Shihao Liang, Yujia Qin, Yining Ye, Xin Cong, Yankai Lin, Yesai Wu, Zhiyuan Liu, and Maosong Sun. Investigate-consolidate-exploit: A general strategy for inter-task agent self-evolution. *arXiv preprint arXiv:2401.13996*, 2024.
- [21] Lifan Yuan, Yangyi Chen, Xingyao Wang, Yi R. Fung, Hao Peng, and Heng Ji. Craft: Customizing llms by creating and retrieving from specialized toolsets, 2024.
- [22] Cheng Qian, Peixuan Han, Qinyu Luo, Bingxiang He, Xiusi Chen, Yuji Zhang, Hongyi Du, Jiarui Yao, Xiaocheng Yang, Denghui Zhang, et al. Escapebench: Pushing language models to think outside the box. *arXiv preprint arXiv:2412.13549*, 2024.
- [23] Hongru Wang, Boyang Xue, Baohang Zhou, Tianhua Zhang, Cunxiang Wang, Huimin Wang, Guanhua Chen, and Kam fai Wong. Self-dc: When to reason and when to act? self divide-and-conquer for compositional unknown questions, 2025.
- [24] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. Toolllm: Facilitating large language models to master 16000+ real-world apis, 2023.
- [25] Cheng Qian, Chenyan Xiong, Zhenghao Liu, and Zhiyuan Liu. Toolink: Linking toolkit creation and using through chain-of-solving on open-source model. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 831–854, 2024.
- [26] Sijia Chen, Yibo Wang, Yi-Feng Wu, Qing-Guo Chen, Zhao Xu, Weihua Luo, Kaifu Zhang, and Lijun Zhang. Advancing tool-augmented large language models: Integrating insights from errors in inference trees, 2025.
- [27] Jiazhan Feng, Shijue Huang, Xingwei Qu, Ge Zhang, Yujia Qin, Baoquan Zhong, Chengquan Jiang, Jinxin Chi, and Wanjuan Zhong. Retool: Reinforcement learning for strategic tool use in llms, 2025.
- [28] Cheng Qian, Emre Can Acikgoz, Qi He, Hongru Wang, Xiusi Chen, Dilek Hakkani-Tür, Gokhan Tur, and Heng Ji. Toolrl: Reward is all tool learning needs. *arXiv preprint*, 2025.
- [29] Wenjun Li, Dexun Li, Kuicai Dong, Cong Zhang, Hao Zhang, Weiwen Liu, Yasheng Wang, Ruiming Tang, and Yong Liu. Adaptive tool use in large language models with meta-cognition trigger, 2025.

- [30] Adam Daniel Laud. *Theory and application of reward shaping in reinforcement learning*. University of Illinois at Urbana-Champaign, 2004.
- [31] Hongru Wang, Huimin Wang, Zezhong Wang, and Kam-Fai Wong. Integrating pretrained language model for dialogue policy evaluation. In *ICASSP 2022 - 2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 6692–6696, 2022.
- [32] Pranjal Aggarwal and Sean Welleck. L1: Controlling how long a reasoning model thinks with reinforcement learning, 2025.
- [33] Daman Arora and Andrea Zanette. Training language models to reason efficiently, 2025.
- [34] Rui Wang, Hongru Wang, Boyang Xue, Jianhui Pang, Shudong Liu, Yi Chen, Jiahao Qiu, Derek Fai Wong, Heng Ji, and Kam-Fai Wong. Harnessing the reasoning economy: A survey of efficient reasoning for large language models, 2025.
- [35] Bairu Hou, Yang Zhang, Jiabao Ji, Yujian Liu, Kaizhi Qian, Jacob Andreas, and Shiyu Chang. Thinkprune: Pruning long chain-of-thought of llms via reinforcement learning, 2025.
- [36] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback, 2022.

A Search Template

Answer the given question. You must conduct reasoning inside `<think>` and `</think>` first every time you get new information. After reasoning, if you find you lack some knowledge, you can call a search engine tool by `<search> query </search>`, and it will return the top searched results between `<observation>` and `</observation>`. **You need to make every search call count and gain helpful results.** If you find no further external knowledge needed, you can directly provide the answer inside `<answer>` and `</answer>` without detailed illustrations. For example, `<answer> xxx </answer>`. Question: question.

B Reward Function

B.1 Figure Illustration

We draw the two types of r_{tool} defined in the main content for better understanding. Figure 6 shows the illustration of these reward functions. It is very clear that: 1) Left: as the number of tool calls increases, the r_{tool} decreases accordingly. Thus when multiple trajectories leads to correct answer, the one that use less tools will get higher reward and the one that do not use tool will get the highest reward; 2) Right: It is obvious that the diagonal achieves the highest reward as the number of tool calls is the optimal one, and as the m increases, the color brightness becomes larger since multiple tool calls are involved.

B.2 Theoretical Justification

We mainly follow the justification (Section 4.2) in the paper [33] to showcase our proposed reward design leads to reducing the tool calls without compromising accuracy. We strongly encourage readers to refer to the previous paper for complete details. All assumptions made in that work still hold in our setting, with the only difference being that our response is defined as $y = (\tau, a)$, which includes both the tool-integrated reasoning trajectory and the final answer, rather than language-only reasoning.

Let θ_{eff}^* denote the population-level parameters of the policy models obtained by maximizing Equation 9, i.e.,

$$\theta_{eff}^* = \arg \max_{\theta} \left\{ \mathbb{E}_{x \sim p} \mathbb{E}_{y \sim p_{\theta}(x)} [(\alpha * r_{tool})] \right\} \quad (10)$$

as $r_{\phi}(q, y)$ is mainly the indicator function about the correctness and format. Therefore, the population-level maximizer $p_{\theta_{eff}^*}$ is as accurate as the population-level maximizer p_{θ}^* and $\text{Acc}(p_{\theta_{eff}^*}^*) = 1$.

B.3 Generalization of Reward

We use two major forms of r_{ϕ} : i) $r_{\phi}^1 = r_{correct}$; and ii) $r_{\phi}^2 = r_{correct} + r_{format}$ to illustrate the generalization of our proposed tool-integrated reward design $r_{\phi}^{tool} = \alpha * r_{tool} * r_{\phi}$.

If the answer is correct, the r_{ϕ}^1 and r_{ϕ}^2 are both positive and fixed for all questions, therefore the r_{ϕ}^{tool} is only determined by the number of α and r_{tool} . Thus it can assign higher score for less tools and lower score for more tools by definition of r_{tool} .

If the answer is wrong, the r_{ϕ}^1 will be 0, and therefore disables the influence of r_{tool} , reducing the reward hacking issue. The r_{ϕ}^2 will be r_{format} and then r_{ϕ}^{tool} becomes $\alpha * r_{tool} * r_{format}$ which is acceptable. Considering two cases of r_{format} , if r_{format} is 0, then the final reward is also 0; and if r_{format} is a positive, the r_{ϕ}^{tool} still holds as less tools will be assigned more rewards. This is reasonable since less tool calls means less cost especially when the answer is wrong. We note that the reward gap here between wrong answer (i.e., r_{format}) and correct answer (i.e., $r_{correct} + r_{format}$) is significant for the policy model to learn the desirable behavior.

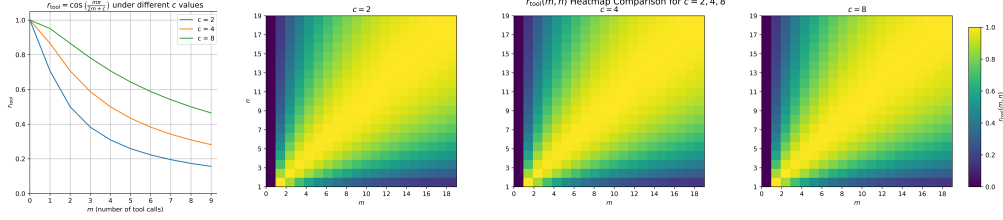


Figure 6: The illustration of different tool-use reward functions for OTC-PPO and OTC-GRPO.

Method	NQ		HotpotQA		TriviaQA		PopQA		2Wiki		Musique		Bamboogle	
	LT	MT	LT	MT	LT	MT	LT	MT	LT	MT	LT	MT	LT	MT
Qwen2.5-3B(-Base)														
OTC-PPO	2,294	0	4,412	0	6,101	0	7,951	0	7,312	15	1,926	0	45	0
OTC-GRPO	2,362	1	2,538	104	5,850	2	7,662	2	2,802	225	789	42	18	1
Qwen2.5-7B(-Base)														
OTC-PPO	3,110	0	5,820	0	10,852	0	12,347	0	9,566	0	1990	0	46	0
OTC-GRPO	2,962	0	5,693	0	10,678	0	11,949	0	9,681	0	2,034	0	48	0

Table 5: Analysis of tool overuse and underuse of OTC-PO against Search-R1. The size of evaluation datasets is NQ (3,610), HotpotQA (7,142), TriviaQA (10,852), PopQA (12,347), 2Wiki (12,576), Musique (2,292), Bamboogle (64).

C Analysis

C.1 Tool Overuse v.s. Tool Underuse

We define two metrics: i) the number of test cases that our method can provide same answer with baseline but using less tools (i.e., LT); and ii) the number of test cases that our method can provide correct answer by using more tools while the baseline can not answer it correctly (i.e., MT). Table 5 shows the results. *It is observed that smaller LLMs tend to underuse tools, while larger LLMs tend to overuse them when reinforcement learning optimization focuses solely on answer correctness.* Specifically, we find our method not only can use less tools to arrive the same answer with the baseline but also can use more tools to provide correct answers, especially on small LLMs. When the model size get larger, it is found that MT always is 0, serving as another evidence that large LLMs over-rely on external tools even when it can not provide correct answer.

C.2 More Examples

Case Study: No Tool Call Required

Question: The Bellagio resort owned by MGM Resorts international is located in which US city?

Answer: Las Vegas

Search-R1: <think> I need to find the US city where the Bellagio resort owned by MGM Resorts International is located. </think>

<search> Bellagio resort owned by MGM Resorts International </search>

<observation> ... </observation>

<search> Bellagio resort city </search>

<observation> ... </observation>

<think> Based on the observations, the Bellagio resort owned by MGM Resorts International is located in Las Vegas, Nevada. </think>

<answer> Las Vegas </answer> (**Tool Calls = 2**)

OTC-GRPO: To answer this question, let's first reason through the information provided:

1. We need to find the US city where the Bellagio resort, owned by MGM Resorts International, is located.
2. The Bellagio resort is a well-known hotel and casino located in Las Vegas, Nevada.
3. Therefore, the US city where the Bellagio resort is located is Las Vegas.

<answer> Las Vegas </answer> (**Tool Calls = 0**)

Figure 7: An example that OTC-GRPO provide correct answer solely rely on its own internal reasoning without any external tool calls while Search-R1 extensively call unnecessary tools.

Case Study: Less Tool Calls with Accurate Search

Question: What science fantasy young adult series, told in first person, has a set of companion books narrating the stories of enslaved worlds and alien species?

Answer: Animorphs

Search-R1: <think> I need to find a science fantasy young adult series that is told in first person and has a set of companion books narrating the stories of enslaved worlds and alien species. </think>

<search> science fantasy young adult series told in first person </search>

<observation> ... </observation>

<search> companion books animorphs enslaved worlds alien species </search>

<observation> ... </observation>

<search> science fantasy young adult series told in first person enslaved worlds alien species companion books </search>

<observation> ... </observation>

<think> Based on the observations, the science fantasy young adult series told in first person and has a set of companion books narrating the stories of enslaved worlds and alien species is "Animorphs". </think>

<answer> Animorphs </answer> (**Tool Calls = 3**)

OTC-GRPO: To answer this question, let's first reason through the information provided:

1. We need to find a science fantasy young adult series.
 2. The series is told in first person.
 3. It has a set of companion books narrating the stories of enslaved worlds and alien species.
- Let's search for this information:

<search> What science fantasy young adult series, told in first person, has a set of companion books narrating the stories of enslaved worlds and alien species? </search>

<observation> ... </observation>

Based on the search results, the science fantasy young adult series that fits the description is "Animorphs".

<answer> Animorphs </answer> (**Tool Calls = 1**)

Figure 8: An example that OTC-GRPO uses less tool calls but with more accurate search to arrive a correct answer.